

Position Distributor coding test

Problem statement

As a market maker, we trade on multiple exchanges. From a risk management point of view, we need to maintain a consistent view of positions across different strategies. For example, if we are trading on Binance, Huobi, and KuCoin, where each strategy is a different process, each of them should have a consistent view of positions across all strategies. We would like you to implement a position distributor that accomplishes this

Data Structures

```
struct symbol_pos {  
    std::string symbol;  
    double net_position;  
};
```

Requirements

1. **Correctness:** At any point in time, all strategies (clients) should have a consistent view of positions from different strategies.
2. **Order Preservation:** Ensure that the position information is processed in the order sent by the original strategy. Altering it would have serious ramifications.
3. **Resilience:** Account for potential information loss (e.g., dropped connections, process crashes) in the design.

As we are an HFT desk, latency is quite important too, but let's consider this an additional bonus compared to 1 & 2.

Please create a github repository and share it with us. Make sure README contains how to build, test this out clearly.

FAQ

1. There are multiple ways to do this. which one to choose?
 - a. We left that out intentionally and let the candidate choose the best mechanism that suits the above requirements. There's no single correct answer for this.
 - i. One way is to have a centralised position_server which collects positions from all different strategies (clients) and distribute it to others
 - ii. Another way would be all strategies (clients) are connected to each other and exchange information
 - iii. Mode of communication can be TCP, UDP or Shared Memory. We leave the choice to the user

Make sure your choices satisfy those above requirements

2. Does it need to be a working version?
 - a. Yes
3. Which language that this needs to be implemented in?
 - a. C++ (anything above 17). You could also use third party libraries if needed but again make sure to put in the README on how to install and compile them
4. How does the task evaluation happen?
 - a. The above three requirements need to be met. Apart from that, we are flexible. We don't have any hard test suite on our side to evaluate this. Once again, we are most interested in your design decisions
5. Do I have to use new cool c++ features?
 - a. Nope. Not necessary. Let's keep it simple
6. Can I modify the data structure provided?

- a. Oh yes. please go ahead. And also feel free to create any other additional modules / data structures to support this

Example Implementation

One way to accomplish this is by running a position server, which collects positions from all clients and broadcasts them to others:

1. **Position Server:** It can use either TCP or UDP or Shared Memory. It reads messages from clients and distributes them accordingly.
2. **Position Client:** Each client feeds its own positions to the position server but also listens for incoming positions from other clients.

For testing purposes, you can run one client by feeding it with some dummy randomly generated data and observe if it appears on other clients. Make sure to differentiate between different client data; for example, the Binance position client can have symbol names like `BTCUSDT.BN`, while the Huobi position client can be `BTCUSDT.HB`.